



ORACLE

API Gatewayご紹介

日本オラクル株式会社

2025年5月



API Gateway概要

近年ますます広がり続けるAPIの世界

ビジネスとビジネスを繋ぎ、拡大するAPIエコノミー

APIの公開は企業やサービスの価値に直接的に影響を及ぼすようになっている

金融機関のAPI公開によるFintechの拡大

- FinTech事業者は、家計簿アプリや貯金アプリなど幅広いサービスを提供

倉庫や配送などの物流API公開

- 物流APIを活用し、利用者は常に最新の配送状況がわかる

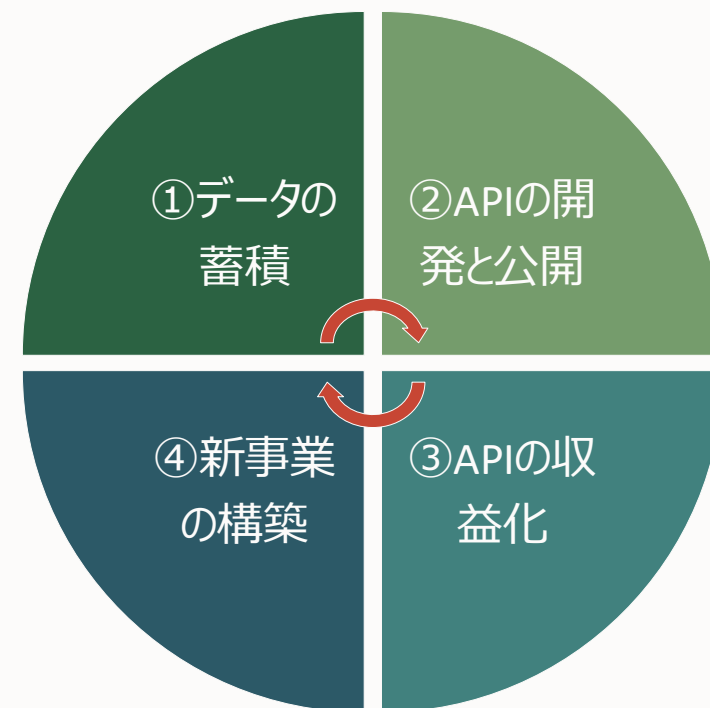
自動車業界にも広がるAPI

- 渋滞情報や走行データのAPI化により、MaaS事業や自動運転の実現に繋がる

政府保有情報のオープンデータ化

- 各自治体や公共事業者は、政府統計のAPIを活用してサービスを構築可能

デジタルビジネスを拡大するためには、APIを活用した事業サイクルは不可欠

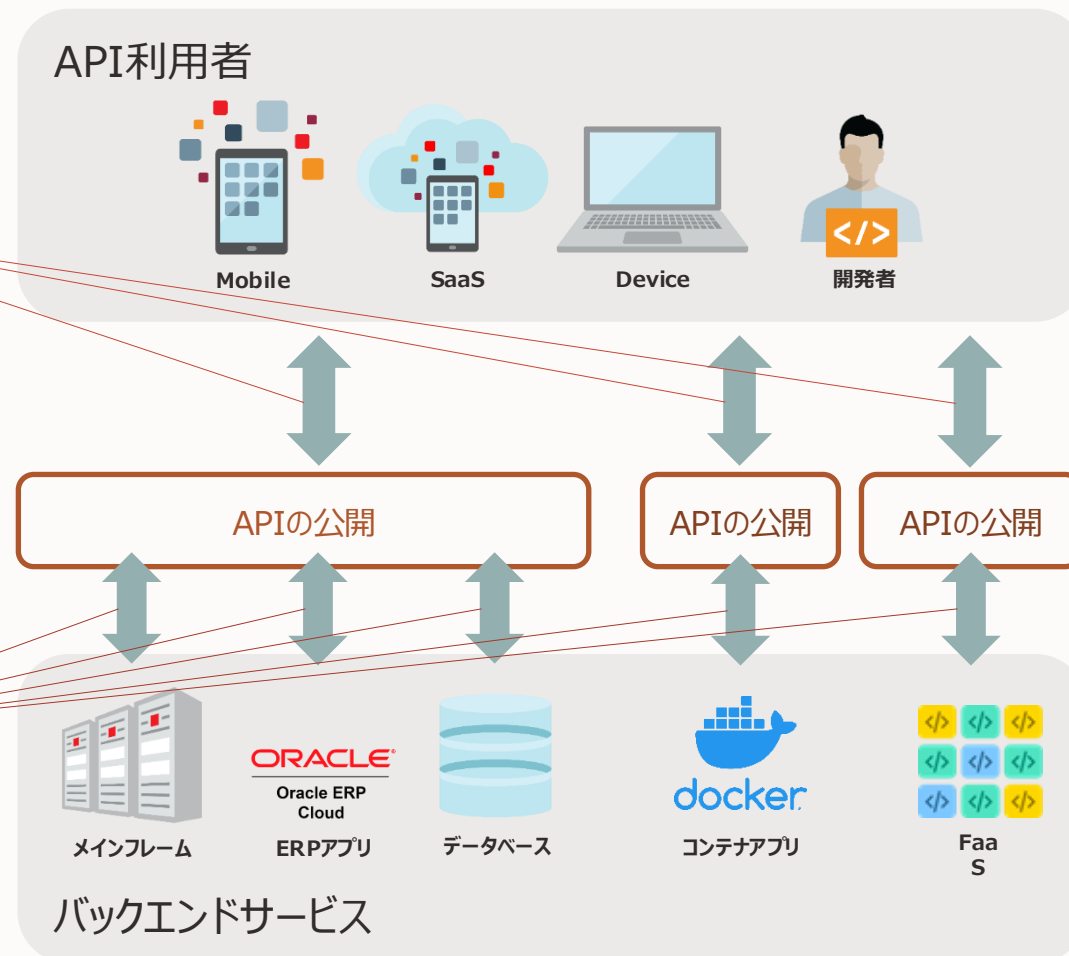


拡がるAPIに反して、煩雑化するその管理

APIの利用/公開の両方で機能する「玄関」が必要

- 複数の認証方法とアクセスポイントがある場合、APIを利用する際の実装や管理が複雑

- 公開するAPIのバックエンドごとに、サービスの監視やライフサイクル管理が発生
- APIを横通ししてセキュリティレベルの維持や利用状況の把握・制御を行うことが困難

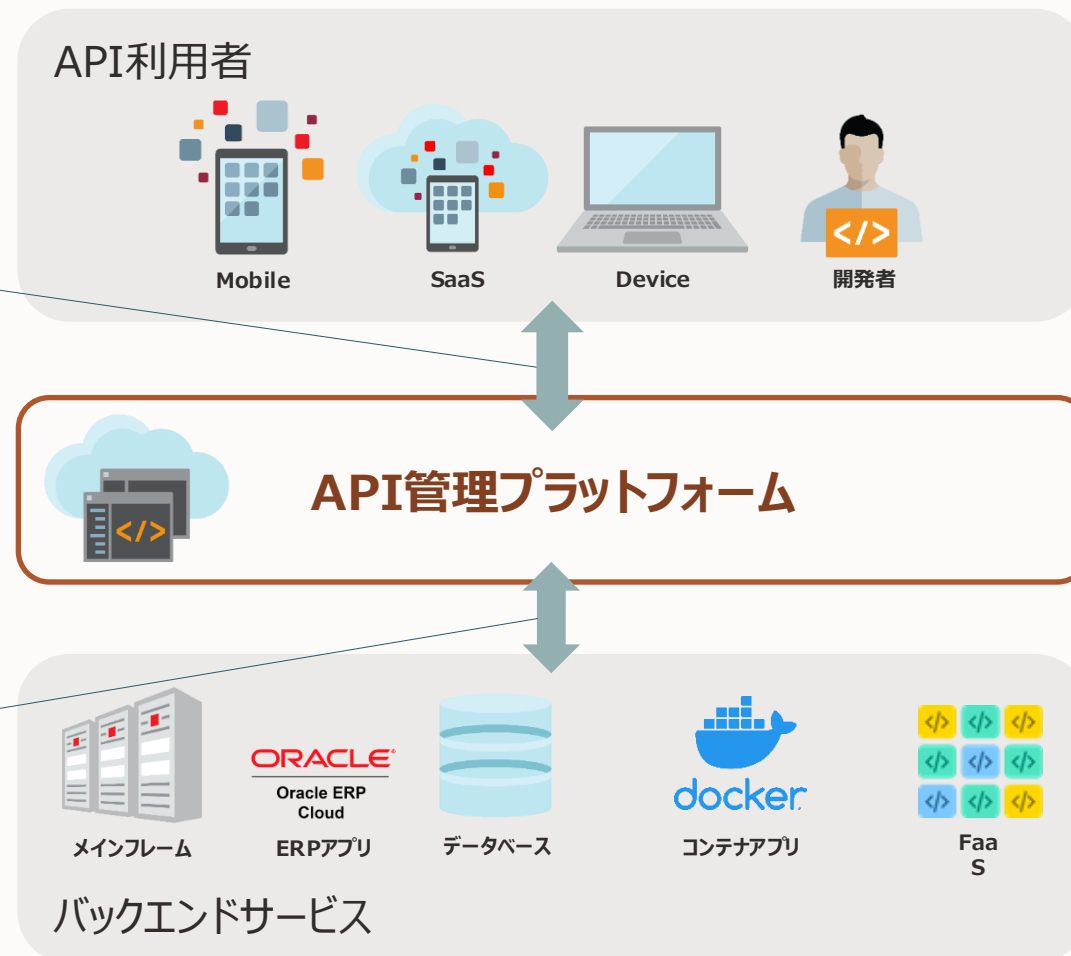


API管理プラットフォームがもたらす、管理のシンプル化

プラットフォームを利用することによる、API管理の一元化

認証方法やアクセスポイントが統一されており、APIを利用する側の実装や管理がシンプルに

APIサービスの監視先やライフサイクルの管理先を一元化



より重要になっていくAPIの管理

APIエコノミーに参加、活用する上で求められるもの



堅牢性/セキュリティ

- 認証/認可
- 流入制限
 - インバウンド/アウトバウンドアクセスの量を制限



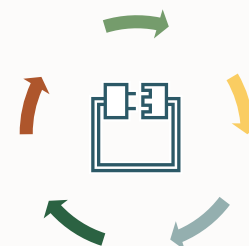
可観測性

- モニタリング
- ロギング
 - アクセスやバックエンド実行の監視/観測が必要



APIデザインの管理

- 開発者ポータル
- ドキュメント
- OpenAPI仕様への対応によるAPIの標準化



ライフサイクル管理

- 設計
- 開発/公開
- バージョニング

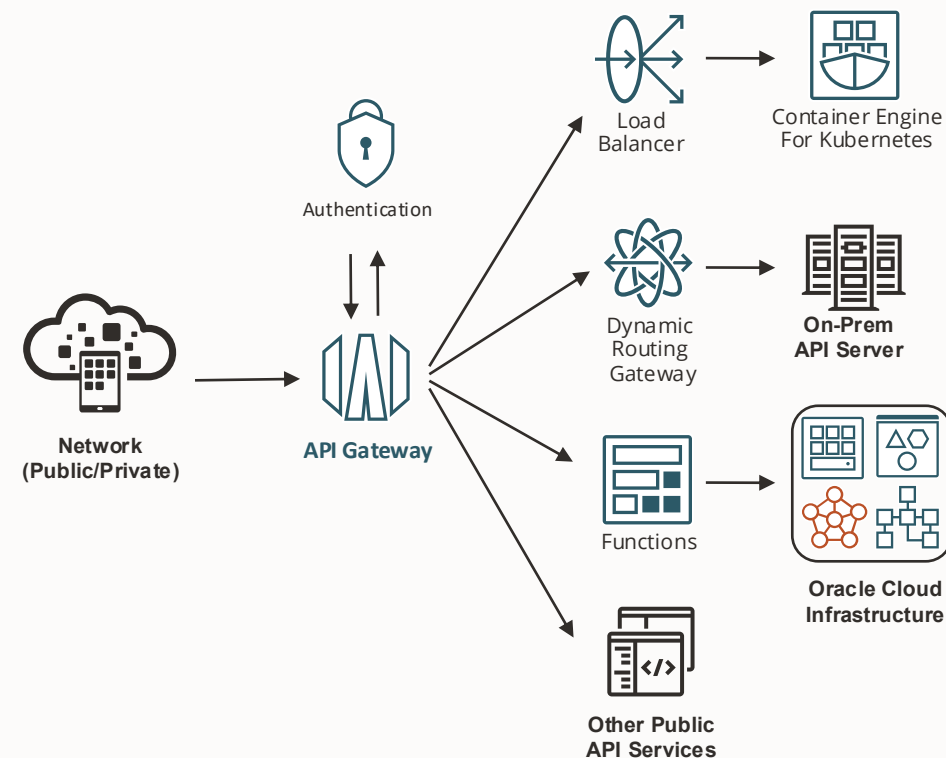


API Gateway

バックエンドサービスの入り口として機能するセキュアなAPIエンドポイント

主な機能 ※それぞれの詳細については後ほど紹介します

- Gateway機能
 - カスタムドメイン/TLS証明書
 - ヘッダー/クエリ変換
 - リザルトキャッシュ(OCI Cache との統合)
 - API利用管理
- 堅牢性の提供
 - カスタム認証
 - JWT検証
 - リクエスト検証
 - 流入制限
 - CORSサポート
- ゲートウェイやAPIの監視
 - OCI Logging, OCI Monitoringとの統合
- ライフサイクル管理
 - GUI/JSONなど用途に合わせたGatewayの操作
 - モックサーバ(Stock Response)の提供
 - SDK生成
- デザイン管理
 - OpenAPI 2.0/3.0のサポート



■ 価格
1,000,000 APIコールごとに¥465

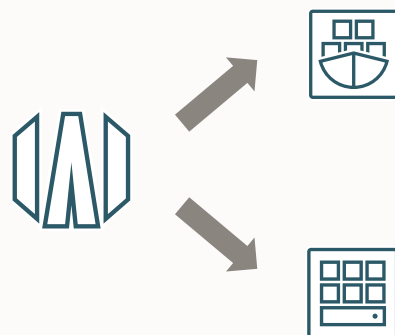


OCI API Gatewayのユースケース

シンプルでセキュアなAPIを簡単に公開

OCIリソースのAPIエンドポイント

- OKEやComputeなどに実装した複数のAPIサーバのエンドポイントを統一化



OCI FunctionsのAPIエンドポイント

- OCI Functionsを社内外に公開できる
- システムやアプリケーションにFunctionsを組み込むことが可能



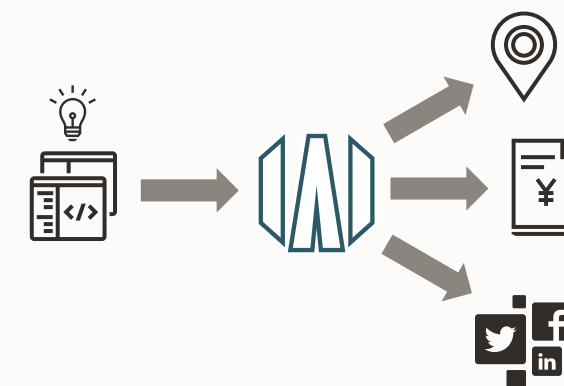
オンプレミスAPIサーバとの接続

- FastConnectやDRGを利用し、既存のオンプレミスAPIサーバをそのまま利用可能



3rdパーティAPIと組み合わせたソリューションの構築

- 外部のAPIサービスもまとめて管理し、自由なシステム構成が実現できる

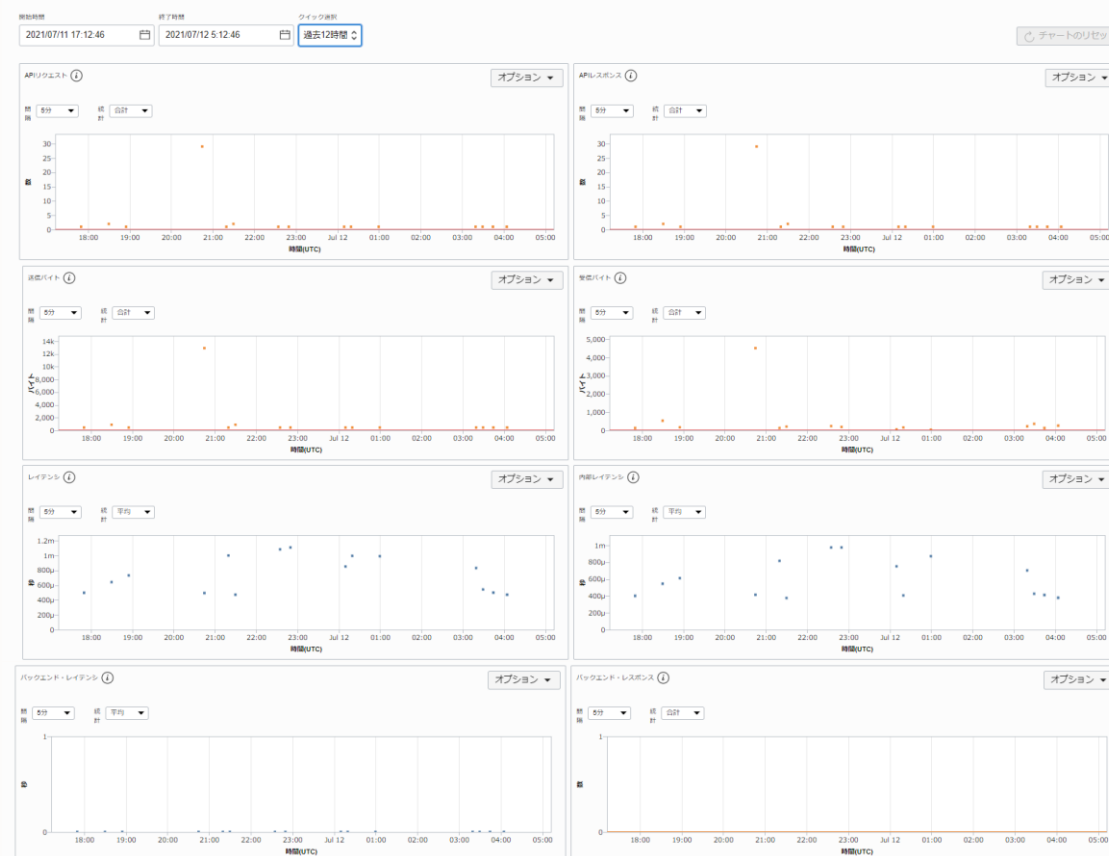


API Gatewayとリソースの監視

OCI Monitoringとの統合

- OCI Monitoringとネイティブに統合される
 - “oci_apigateway”というメトリック・ネームスペースでメトリックの収集が行われる
- デフォルトで使用可能なメトリック
 - BytesReceived
 - BytesSent
 - HttpRequests
 - HttpResponses
 - BackendHttpResponses
 - Latency
 - IntegrationLatency
 - InternalLatency

メトリック



OCI Loggingとの統合

- OCI Loggingとネイティブに統合される
 - デプロイメント単位ごとにログ出力の要否が設定可能
 - OCI Loggingのインターフェースを利用して閲覧
- Service Connector Hubと併用することでログ出力を元にした統合的な運用を可能に
- 以下の2種類のログが出力可能
 - アクセスログ: API Gatewayを介してデプロイメントとの間で送受信されるリクエスト/レスポンスデータのサマリー
 - 実行ログ: API Gateway内の処理に関するリクエスト/レスポンスの実行ログ

デプロイメント単位にログ出力を有効化

ログ

カテゴリ	ステータス	ログ名	ログ・グループ	ログの有効化
Access Logs	● 作成中	greeting2_access	Function_Log_Group	☒ 有効
Execution Logs	● 作成中	greeting2_execution	Function_Log_Group	☒ 有効

アクセスログ

ログの探索



実行ログ

ログの探索



API Gatewayとリソースの保護

カスタム・ドメインとTLS証明書

Oracle指定の認証局から発行されたTLS証明書ではなく、ユーザーが取得したTLS証明書が使用可能

証明書の取得先	ドメイン名	Public IP との紐づけ	証明書の 有効期限と更新処理
Oracle指定の認証局 (デフォルト)	<unique>.apigateway.<region>.oci.customer-oci.com	自動	期限切れとなる前に自動更新
証明書サービス	独自ドメイン	ユーザーが実施	期限切れとなる前にユーザーが更新
サード・パーティ	独自ドメイン	ユーザーが実施	期限切れとなる前にユーザーが更新



TLS証明書検証のためのトラスト・ストアのカスタマイズ

- デフォルトのCAバンドルに加えて、証明書サービスで作成した他のCA（カスタムCA）のルート証明書および他のCAバンドル（カスタムCAバンドル）をAPI Gatewayのトラスト・ストアに追加可能
- ユースケース
 - セキュリティ上の理由でプライベートなCAのみを含むカスタムトラスト・ストアを使いたい
 - mTLS(相互TLS)をサポートしたい

認証局の追加

認証局

認証局(CA)バンドルは、TLS証明書の信頼性を検証するために、ゲートウェイで使用されます。

タイプ

認証局

証明書バンドルのタイプを指定します

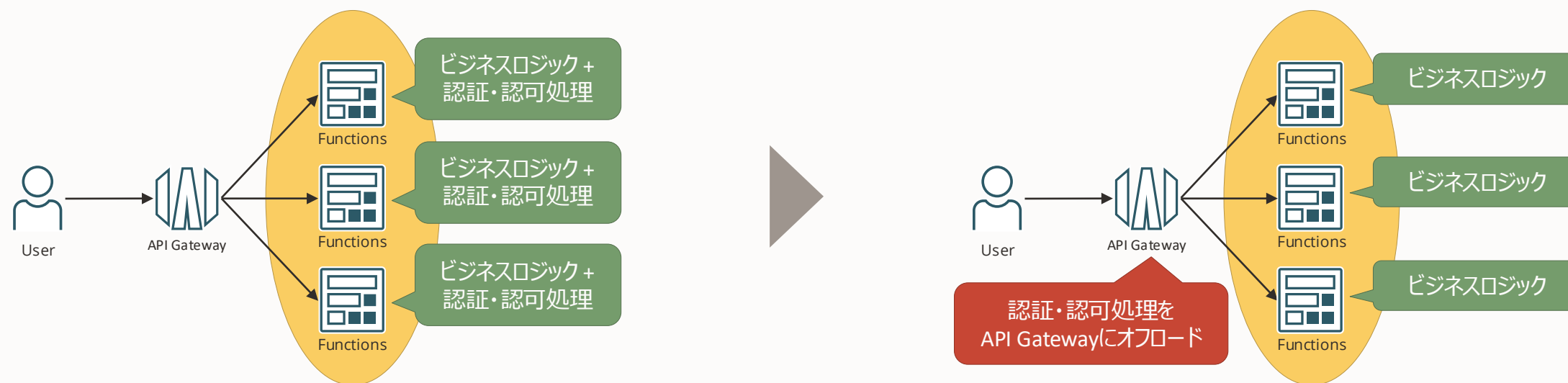
の証明書バンドル/認証局 [\(コンパートメントの変更\)](#)

-private-ca

+ 追加の認証局

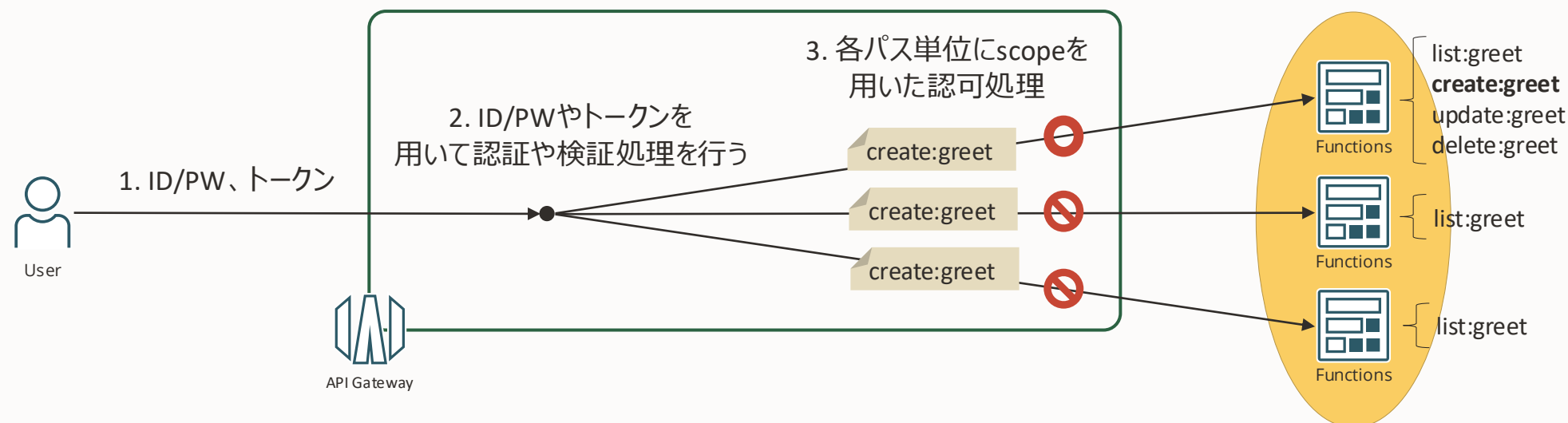
API Gatewayを活用したWeb APIに対する認証・認可処理

- API Gatewayの機能を活用することで、各アプリケーションに実装していた認証・認可処理をオフロード
- バックエンド実行基盤は、よりビジネスロジックに集中できる
- API Gatewayで一元管理することで、認証強度や鍵のライフサイクル管理を統一できる



API Gatewayで行われる認証・認可処理の基本的な動き

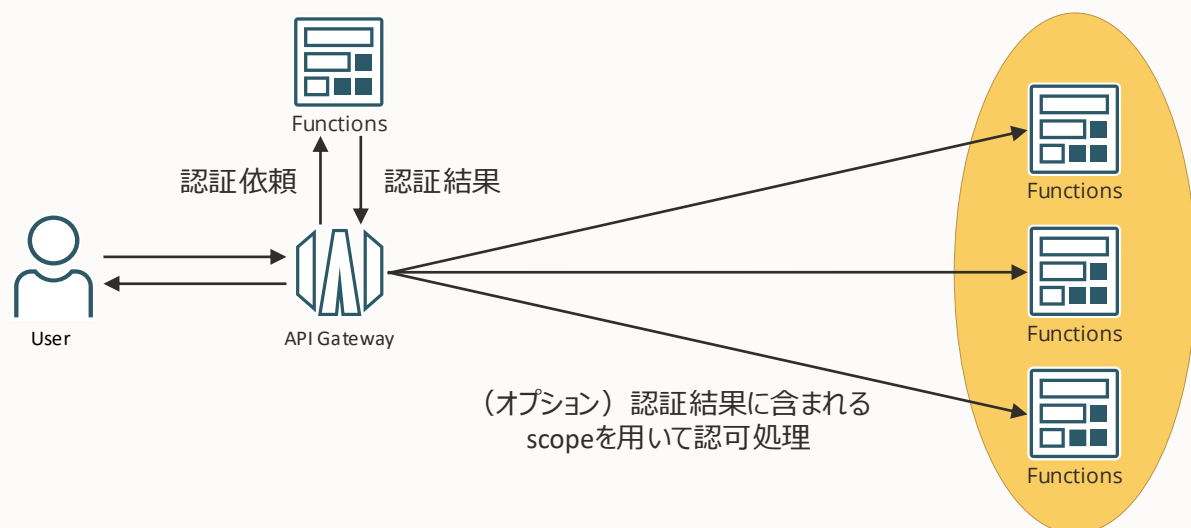
1. エンドユーザーが認証・認可のために必要な情報（ID/Password, トークン, etc.）含めてリクエストを送信
2. API Gatewayにて、認証やトークンの検証処理が行われる。方法は以下の2通り。
 - Oracle Functionsで実装したロジックを用いて、認証を行う → **認可プロバイダ・ファンクション**
 - 設定値に基づき、トークンの検証を行う → **OAuth 2.0/OpenID Connect 1.0**
3. 認証結果、検証済みのトークンに含まれるscopeを用いて、各パス単位に認可処理を行う
 - スキップ（認可処理を実行しない）も可能



提供されている認証・認可のための機能

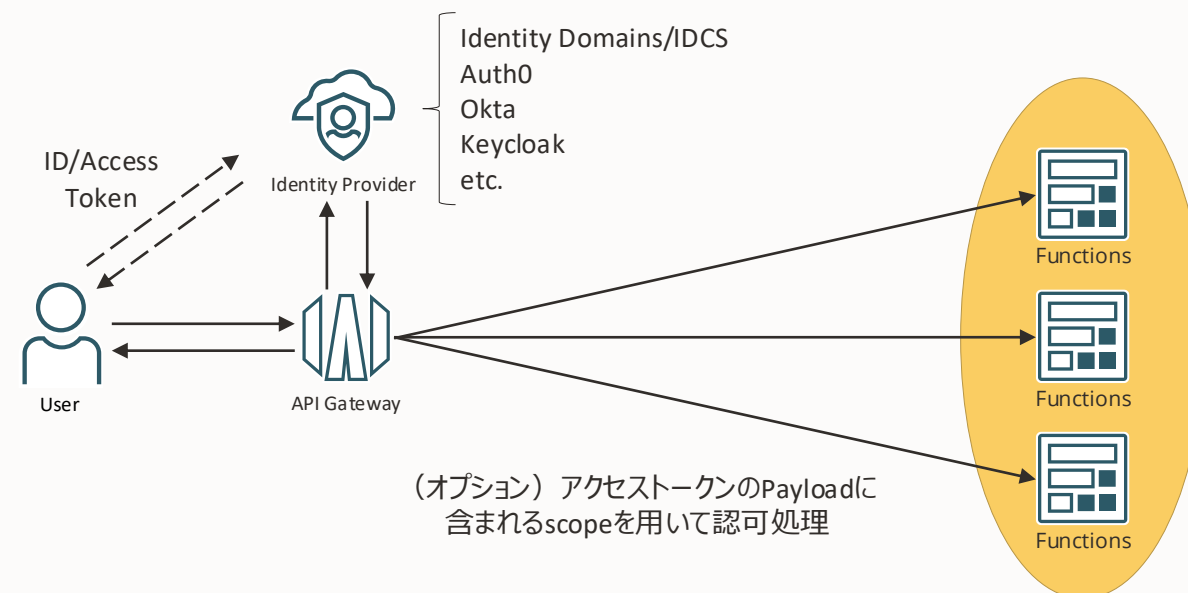
認可プロバイダ・ファンクション

- HTTPリクエスト（ヘッダー or クエリーパラメーター）に含まれる情報で認証処理を行うOracle Functionsへ渡し、その情報を検証する事で認証処理を行う
- （オプション）返却された認証結果に含まれるscopeを用いてパスに対する認可処理を行う



OAuth 2.0/OpenID Connect 1.0

- HTTPリクエスト（ヘッダー or クエリーパラメーター or Cookie）に含まれるID/アクセストークンをAPI Gatewayで検証する
- （オプション）アクセストークンに含まれているscopeを用いて、パスに対する認可処理を行う



CORSサポート

- CORS(Cross-Origin Resource Sharing)
 - あるオリジンで動作しているWebアプリケーションに異なるオリジンに存在するリソースへのアクセス権をHTTPリクエストによって与えるための仕組み
- API Gatewayを使用し、デプロイされるAPIに対するCORSサポートを有効にできる
 - 各API単位の対応が不要になるため、効率的にポリシーの適応が可能

CORSポリシー

ヘルプ

オリジン

許可されるオリジン ⓘ

www.example.com ×

+ 別のオリジン

メソッド

許可されるメソッド オプション ⓘ

* (ワイルドカード) ×

ヘッダー

許可されるヘッダー オプション ⓘ

×

+ 別のヘッダー

公開ヘッダー オプション ⓘ

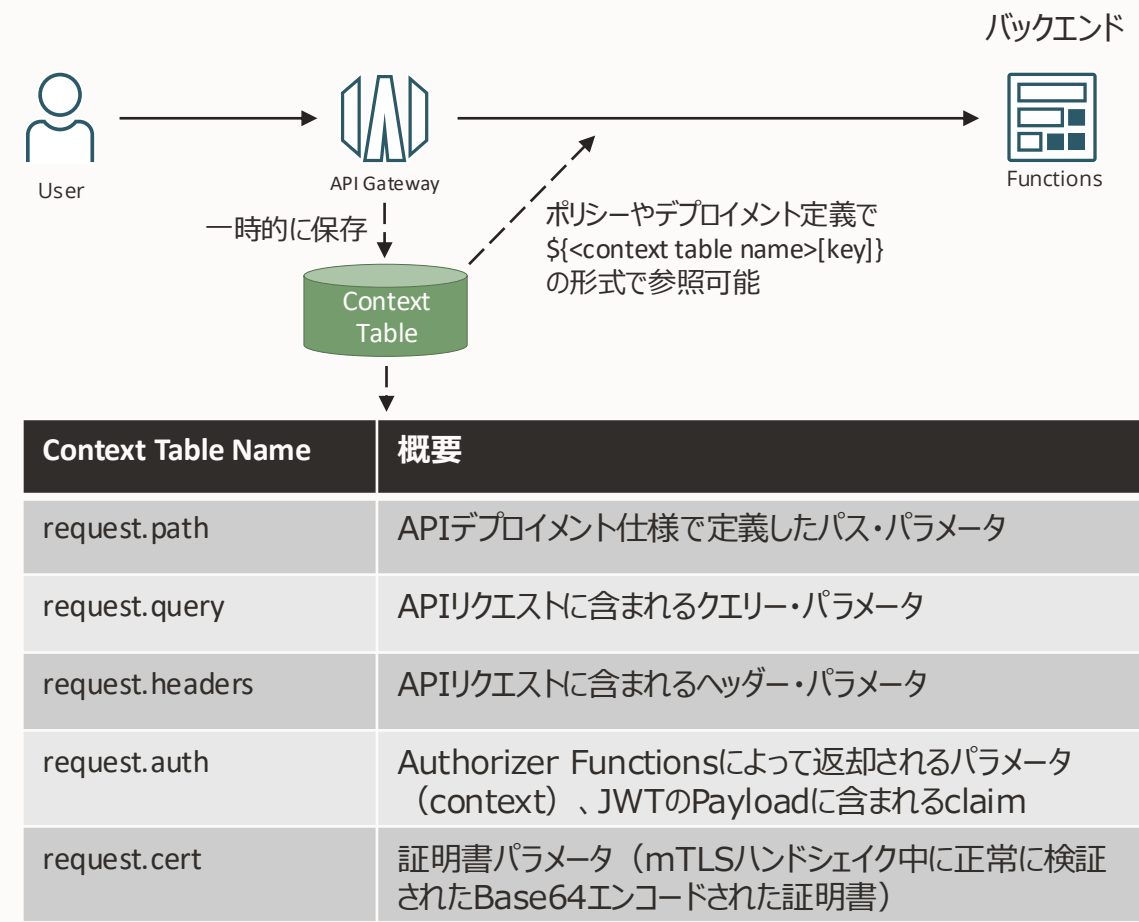
×

+ 別のヘッダー

APIの呼び出しとパラメータ化

コンテキスト変数

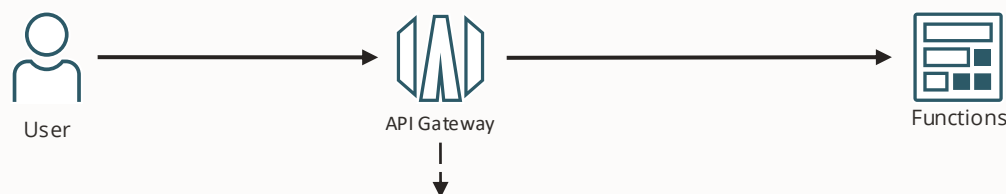
- APIコール等に含まれるパラメータをポリシーやデプロイメント定義で使用可能にするために、API Gatewayが一時的に保持する変数のこと
- ユースケース例
 - バックエンドが解釈できる形へリクエストの形式を変更したい（request.path/queryを用いて、URLの形式を書き換える）
 - 認証結果に含まれる情報（ユーザーID、etc.）をバックエンドに伝搬したい



リクエスト・ポリシーとレスポンス・ポリシー

流入制限

- API Gatewayのバックエンドに対して、グローバルにレート制限を設定可能
 - 過剰なリクエストからバックエンドを保護することで、可用性やリソースの公平な使用を維持する
 - DoS攻撃の防止
 - API使用を収益化するために利用を制限する
 - etc.
- 1秒当たりのリクエスト数の上限を全リクエストの合計 or クライアントIP単位で指定する



レート制限ポリシー [ヘルプ](#)

1秒当たりのリクエスト数

100

レート制限のタイプ

レート・キーを選択

レート・キーを選択

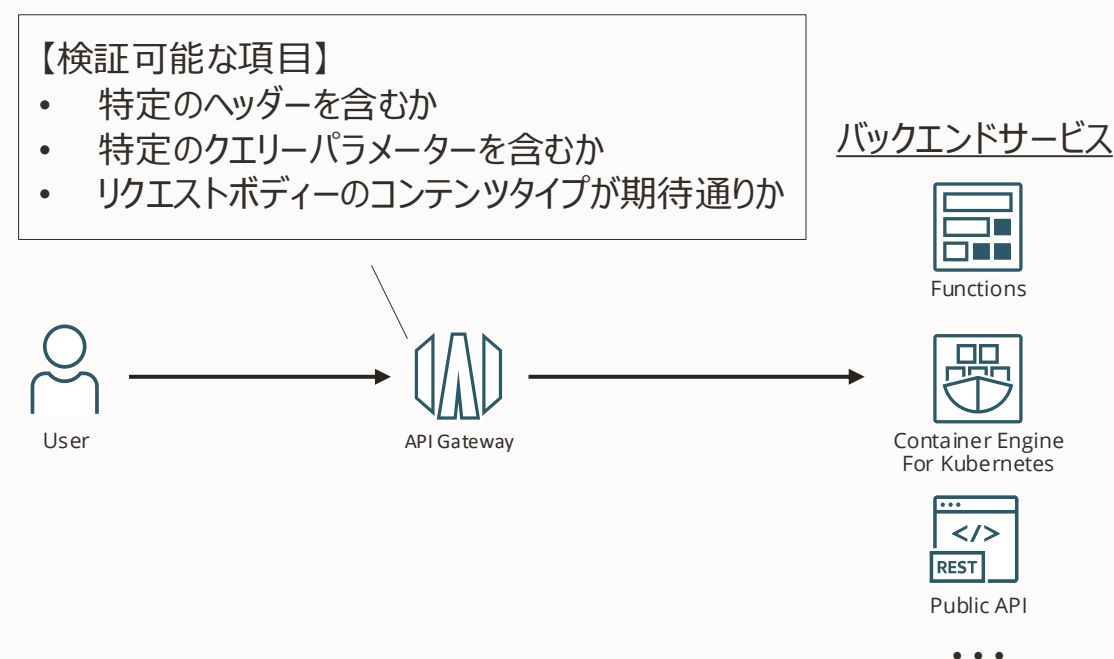
合計

クライアント当たり(IP)



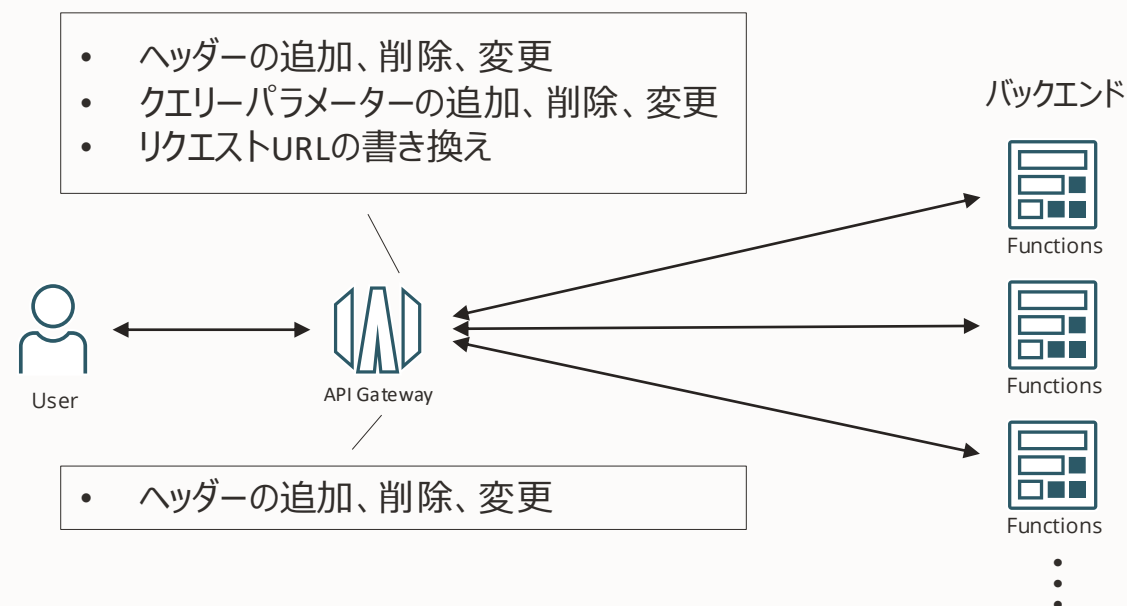
リクエストのバリデーション

- API GatewayでHTTPリクエストに対するバリデーションが可能
- バックエンドサービスに対して不要な負荷をかけないようにするために、HTTPリクエストのバリデーション処理をAPI Gatewayにオフロードする
- 検証のポリシー
 - Enforcing: すべてのリクエストに対して検証を実施し、検証が成功したリクエストのみバックエンドへルーティングする
 - Permissive: すべてのリクエストに対して検証を実施するが、検証結果によらずすべてのリクエストをバックエンドへルーティングする
 - Disabled: 検証を実施しない



ヘッダー／クエリー変換

- API Gatewayで、HTTP Request/Responseの一部の情報に対して変更が可能
- ユースケース
 - バックエンドサービスが要求するヘッダー(e.g. Accept-Encoding)を一律で付与する
 - セキュリティ上の理由で、レスポンスに含まれる特定のヘッダー(e.g. Server)を削除する
 - バックエンドサービスが解釈できるような形にリクエストのURLを書き換える
 - etc.



e.g. Request Headerに対して、上書きの設定を追加をする

リクエスト・ヘッダー変換

すべての変換ポリシーは、定義済みの認証ポリシーおよび認可ポリシーが実施された後に適用されます。リクエスト・ヘッダー変換について[さらに学習](#)します。

アクション (i)	動作 (i)	ヘッダー名	値
設定	上書き	Test-Header1	value x



APIの開発支援機能

OpenAPI 2.0/3.0のサポート

- API GatewayでOpenAPI仕様(2.0 & 3.0)をサポート
- デザイン・ファーストなAPIの開発サイクルに貢献
 - API用のリファレンスドキュメントの共有
 - 開発初期のMockサーバーを容易に生成
 - 認証・認可等のポリシーを追加し、さらに高度なスタブとして活用可能
- OpenAPI仕様を元にAPIクライアント用のSDKが各言語で生成可能
 - Android Java
 - Java
 - JavaScript
 - Swift
 - TypeScript

Create API

NAME: OpenAPI Example

COMPARTMENT: shuhei.kawamura

UPLOAD API DESCRIPTION FILE (SWAGGER, OPENAPI 3.X SPECIFICATION)

Drop a file. [Browse](#)

JSON or YAML format

employee.yaml x

OpenAPI仕様の定義ファイルをアップロードし、APIリソースを作成

APIリソースを使用して、API GatewayにAPIをデプロイ

```

openapi: '3.0.2'
info:
  title: Employee API
  version: '1.0'
servers:
  - url: https://oci-api-gateway-host
paths:
  # ...
  /employee:
    get:
      responses:
        '200':
          description: return all employee.
          # ...
          examples:

```

Route 1

PATH: /employee

METHODS: GET

TYPE: Stock Response

STATUS CODE: 200

BODY: [{"id": 1, "name": "hoge"}, {"id": 2, "name": "fuga"}]



モックサーバー(Stock Response)の提供

- 開発初期のモックサーバー構築が容易
 - APIの内部実装が完成した後は、URLを変更することなく固定値から実APIへ置き換え可能
- OpenAPI仕様から作成したAPIリソースを用いて、APIデプロイメント作成した場合は、デフォルトでモックサーバーとして提供される
- 様々なポリシーと組み合わせることで高度なモックサーバーを構築可能
 - 認証・認可
 - リクエストのバリデーション
 - レート制限
 - ロギング
 - ...

1のルーティング

パス

メソッド

タイプ

バックエンド・サービスのタイプを指定します。標準レスポンスバックエンドについて [さらに学習](#)します。

ステータス・コード

本文 オプション

ヘッダー名 オプション ヘッダー値 オプション

+ 別のヘッダー

> [リクエスト・ポリシーのルーティングを表示](#)

> [レスポンス・ポリシーのルーティングを表示](#)

> [レスポンス・キャッシング・ポリシーを表示](#)

> [ロギング・ポリシーのルーティングを表示](#)



クライアントSDKの生成

- OpenAPI仕様から作成したAPIリソースを元にAPIクライアント用のSDK(Software Development Kits)を作成可能
- 対応言語
 - Android Java
 - Java
 - JavaScript
 - Swift
 - TypeScript

SDKを用いたサンプルコード(Java):

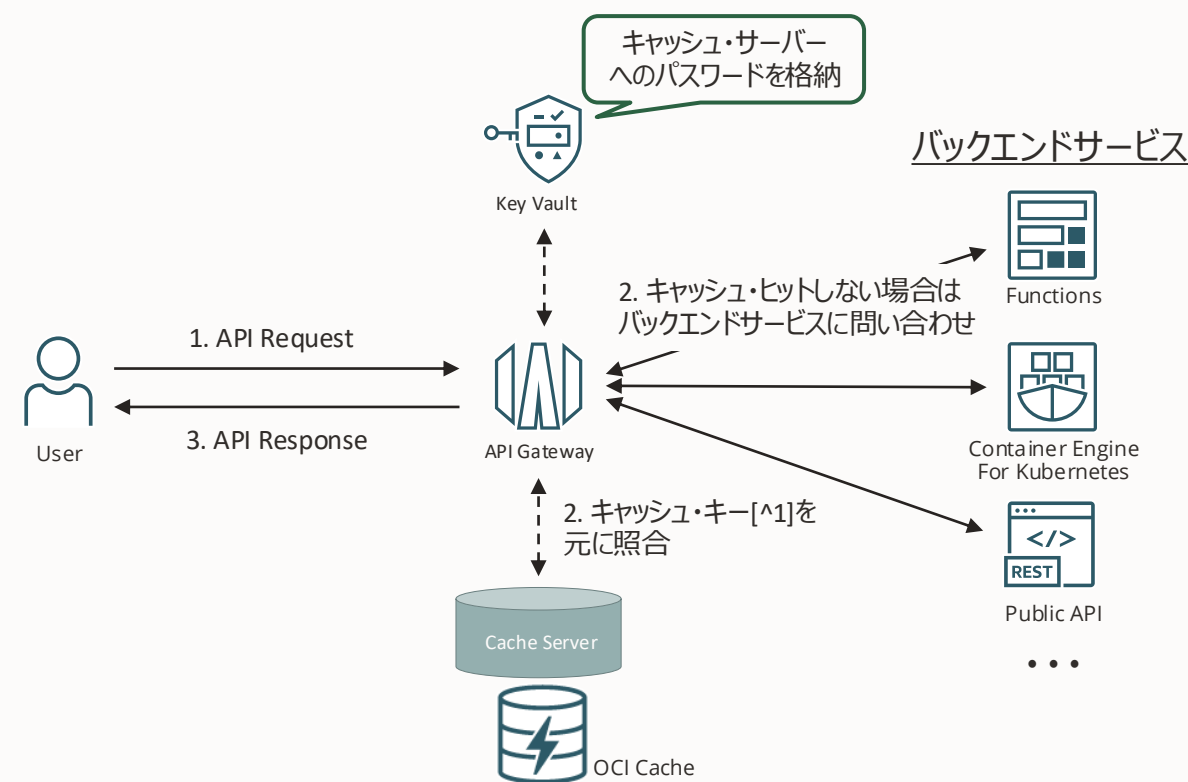
```
// Import classes:
import com.example.sdk.ApiClient;
import com.example.sdk.ApiException;
import com.example.sdk.Configuration;
import com.example.sdk.models.*;
import com.example.sdk.api.DefaultApi;

public class Example {
    public static void main(String[] args) {
        ApiClient defaultClient = Configuration.getDefaultApiClient();
        defaultClient.setBasePath("https://hoyd2...");
        DefaultApi apiInstance = new DefaultApi(defaultClient);
        Integer employeeId = 56; // Integer |
        try {
            apiInstance.employeeEmployeeIdDelete(employeeId);
        } catch (ApiException e) {
            System.err.println("Exception when calling DefaultApi#employeeEmployeeIdDelete");
            System.err.println("Status code: " + e.getStatusCode());
            System.err.println("Reason: " + e.getResponseBody());
            System.err.println("Response headers: " + e.getResponseHeaders());
            e.printStackTrace();
        }
    }
}
```



レスポンス・キャッシュ

- 外部に構築したキャッシュ・サーバーと統合することで、同一リクエスト(GET, HEAD, OPTIONS)に対するレスポンスをキャッシュ・サーバーから返却する機能
- ユースケース
 - 同一リクエストに対する応答を高速化したい
 - バックエンド・サービスに対して必要以上に負荷をかけたくない



[^1]: リクエストのURL、HTTP メソッド(GET, HEAD, OPTIONS)、APIデプロイメントのOCIDから導出されるキャッシュ・サーバー内のレスポンスを一意に識別するためのキー



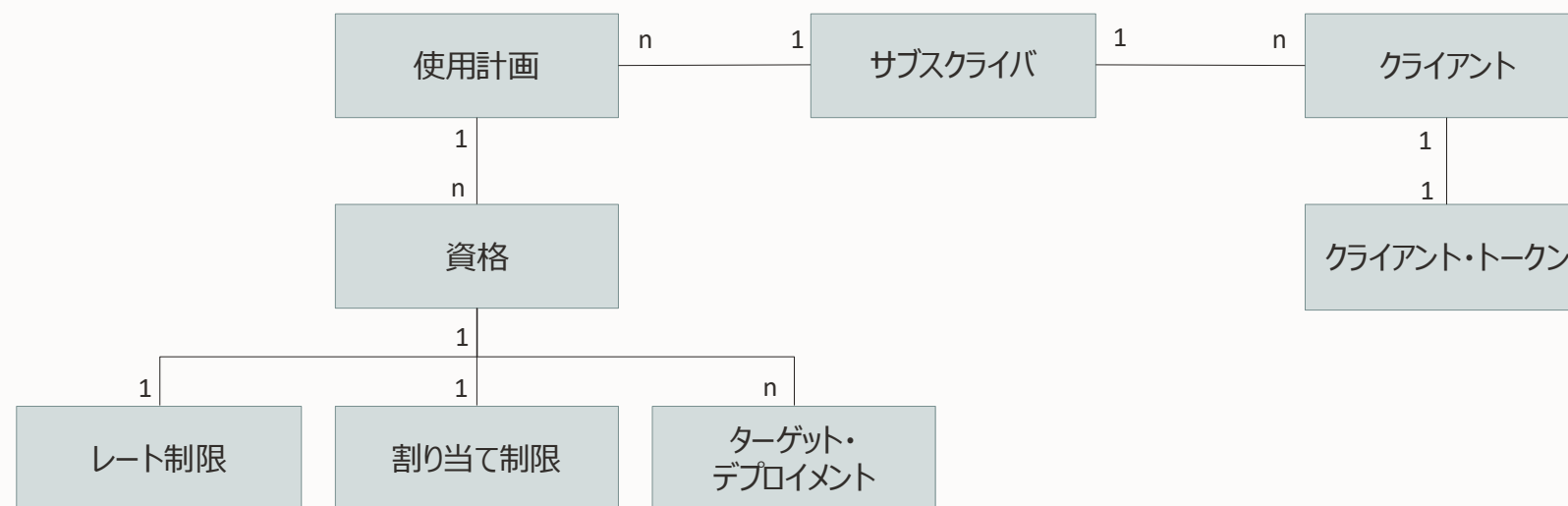
APIの利用管理

使用計画（使用プラン）とサブスクライバ

リソースの全体像

API の使用計画をサブスクライバごとに設定し、利用の制限とメトリックの収集がサブスクライバ単位で可能に
使用計画には、レート制限（1 秒あたり n 回まで）と割り当て制限（分/時間/日/週/月あたり n 回まで）を設けることができる

制限は、クライアント／コンシューマーごとに一意に設定したクライアント・トークンを用いて行われる



使用計画とサブスクライバ

クライアント・トークン

API Gateway はクライアント／コンシューマー毎に払い出したクライアント・トークンを元に使用計画の判定を行う
クライアント／コンシューマーは以下の場所のいずれかにクライアント・トークンを含めてリクエストを発行する

- 認可プロバイダ・ファンクションによって返却される認証パラメータ or JWT のペイロード
- リクエスト・ヘッダー
- クエリーパラメータ
- パス・パラメータ
- リクエストの Host ヘッダー

事例



CLINKS 株式会社様

OCIのコストパフォーマンス、拡張性を活かし、 画期的なサーバレスアーキテクチャを構築

- AWSと比較して全体で**約40%のコスト削減**。またOCI Media Serviceのストレージ使用量ベースの課金と安価なデータ転送費用により、当該部分は**約80%のコスト削減**
- API GatewayとObject Storageで**静的Webサイト・ホスティング機能**を実装。また動画ファイル処理実行にはFunctions、DBとしてはNoSQL Databaseを活用し、**拡張性を意識したサーバレス構成**に
- OCI Data Scienceで動画の**AIリコメンデーションを実現**
- 今後の計画として、MinutesGamesのプラットフォームを基に、小売、教育、ヘルスケアなど多様な業界向けのソリューション開発を企画。小売業向けにはライブコマース機能を強化し、教育分野では対話型学習コンテンツの配信を可能に



The graphic is split into two horizontal sections. The top section has an orange background and features the word 'EXPLORER' in large, bold, white letters. Below it, in smaller white text, is 'ゲーム発見の旅へ！' and '短時間で楽しめるゲーム体験' followed by '動画視聴中にゲームをダウンロード' and 'すぐにゲームをプレイ可能！'. To the right is a black smartphone icon with a white screen showing a red square and a white line graph. The bottom section has a teal background and features the 'MinutesGames' logo on the left, which includes a stylized blue and yellow 'M' icon. To the right of the logo is the word 'CREATOR' in large, bold, white letters. Below it, in smaller white text, is '新しい時代を彩るゲームクリエイターに' and '世界中に「作りたい」を広げて加速' followed by '効率的に心が満たされる遊びのサイクルと' and '夢を広げて「手軽に作る」サイクル'. At the bottom of the teal section is a pixel-art illustration of a person with brown hair sitting at a desk with two computer monitors and a laptop.

<https://app.minutes.games/teaser/>

顧客事例：CLINKS 株式会社様

OCIのコストパフォーマンス、拡張性を活かし、画期的なサーバレスアーキテクチャを構築

MinutesGames

- クリエイターが開発したミニゲームを、ショート動画形式に変換し配信する統合プラットフォーム。ユーザーの視聴履歴や好みに基づいて動画を推奨。「1分で心を満たす」体験を提供し、「ゲーム開発」の再評価を実現

従来の課題

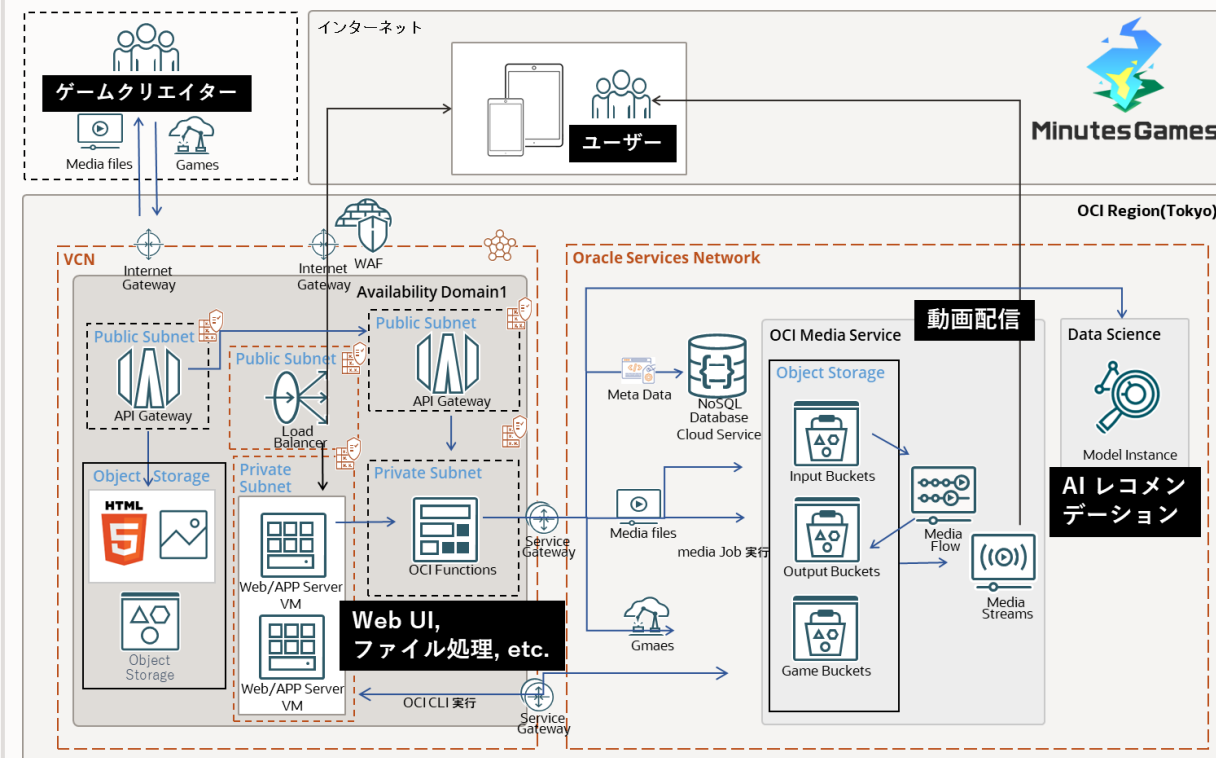
- 大容量の動画変換・ストリーミング配信システムを、コスト効率良く、かつ急激な需要にスケラブルに対応できる構成で構築する必要があった。広範なユーザー層にコンテンツを届けるAIリコメンデーションの仕組みが必要だった

採用ポイントと導入効果

- AWSと比較して全体で**約40%のコスト削減**。またOCI Media Serviceのストレージ使用量ベースの課金と安価なデータ転送費用により、当該部分は**約80%のコスト削減**
- API GatewayとObject Storageで**静的Webサイト・ホスティング**機能を実装。また動画ファイル処理実行にはFunctions、DBとしてはNoSQL Databaseを活用し、**拡張性を意識したサーバレス構成**に
- OCI Data Scienceで動画の**AIリコメンデーションを実現**

システム構成イメージ

<https://app.minutes.games/teaser/>



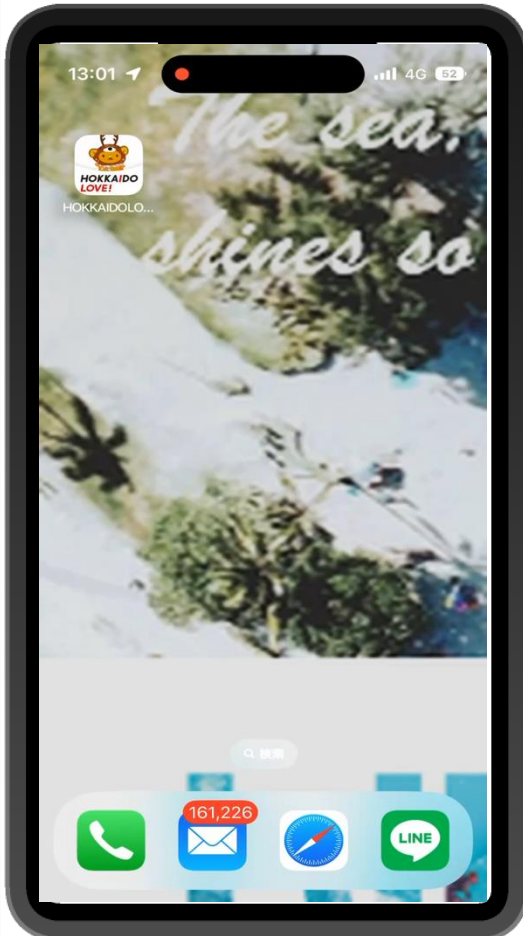
利用サービス

- OCI Media Services / OCI Functions / OCI API Gateway / NoSQL Database Cloud Service / OCI Data Science, etc.





Hokkaido Love! アプリ



<開発のポイント>

- Flutterを活用したマルチOSアプリケーション
- Geo Fencingを利用したバックグラウンドの位置情報取得
- リアルタイムポイント処理の実装をAPI Gateway + Functionsで実装



<効果>

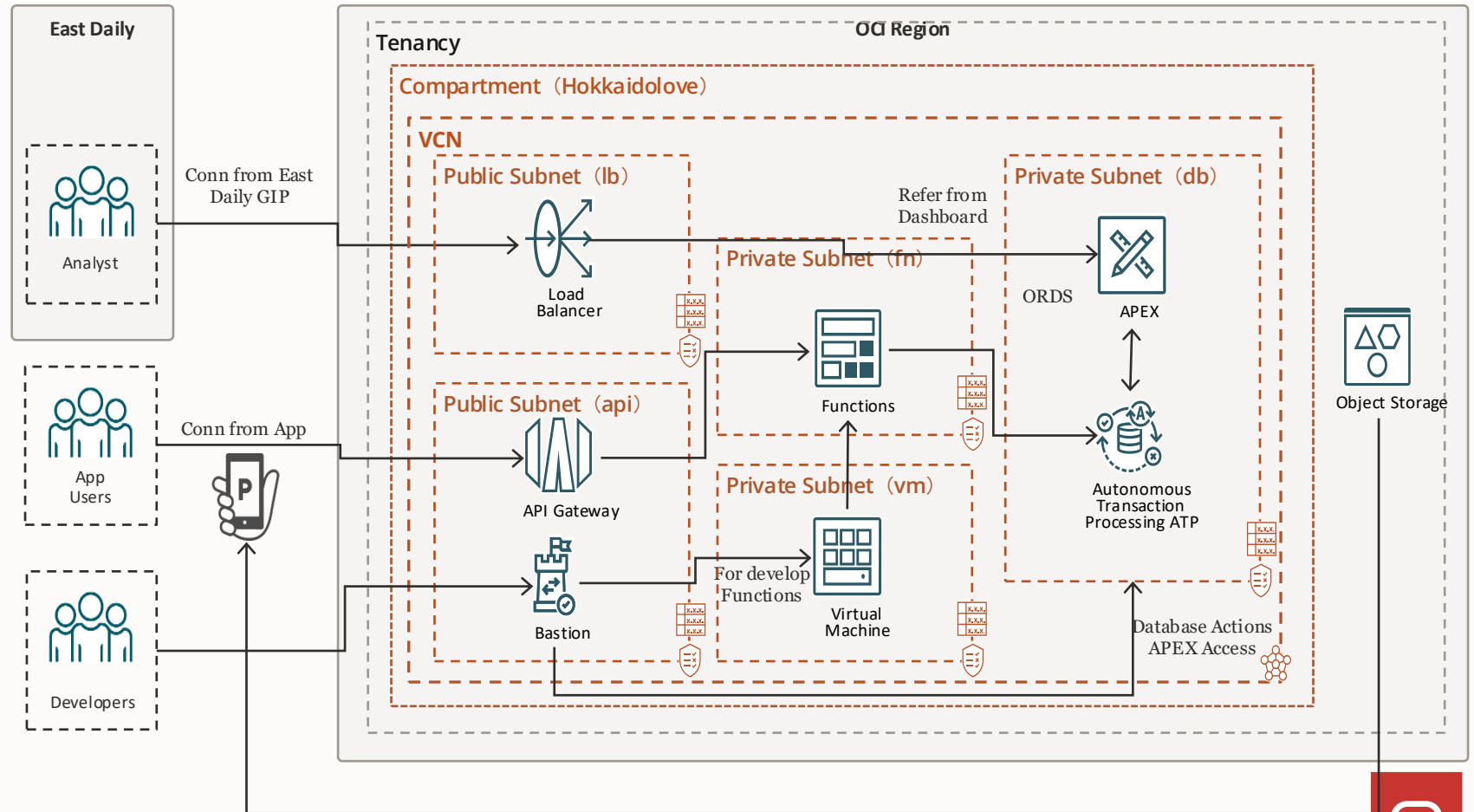
- **超短期間(3ヵ月)での実装**を実現
- 競合に比べて、およそ**1/3程度での初期コスト**
- アプリ開発と同時に**分析ダッシュボード**も提供

Hokkaido Love! プラットフォーム

Direction1.
抜群の費用対効果

Direction2.
**利用状況に応じた
柔軟なスケーラビリティ**

Direction3.
低価格大容量ストレージ



ORACLE